

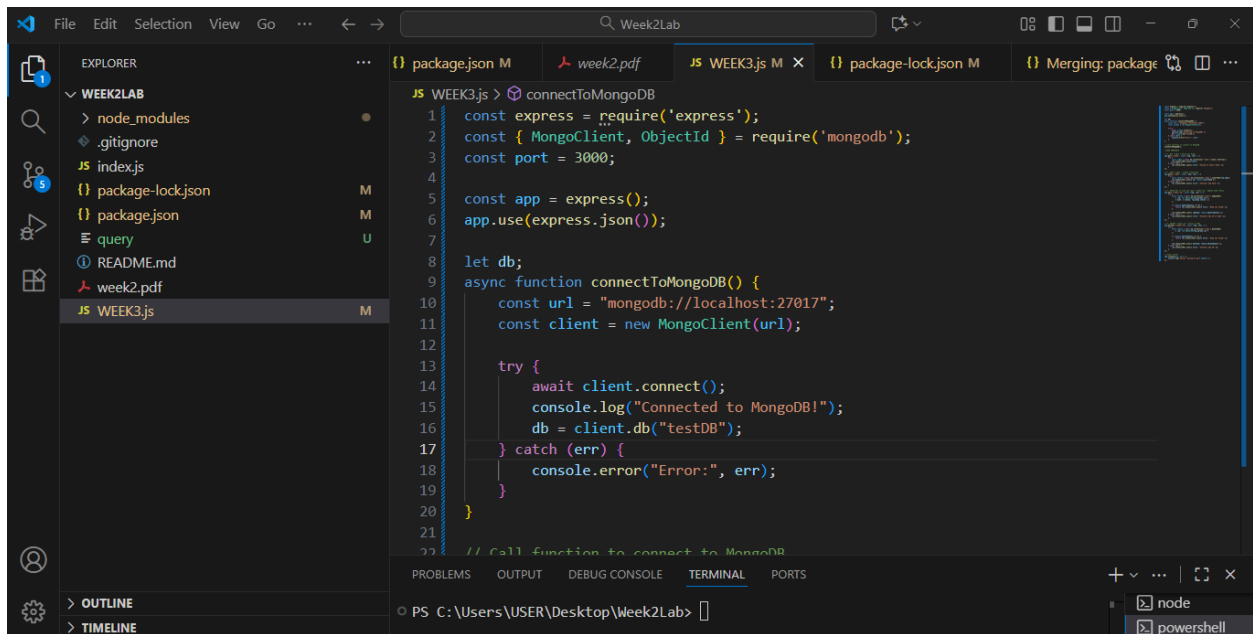


Figure 1: Week 3 Exercise: Building a Ride-Hailing REST API with Express.js

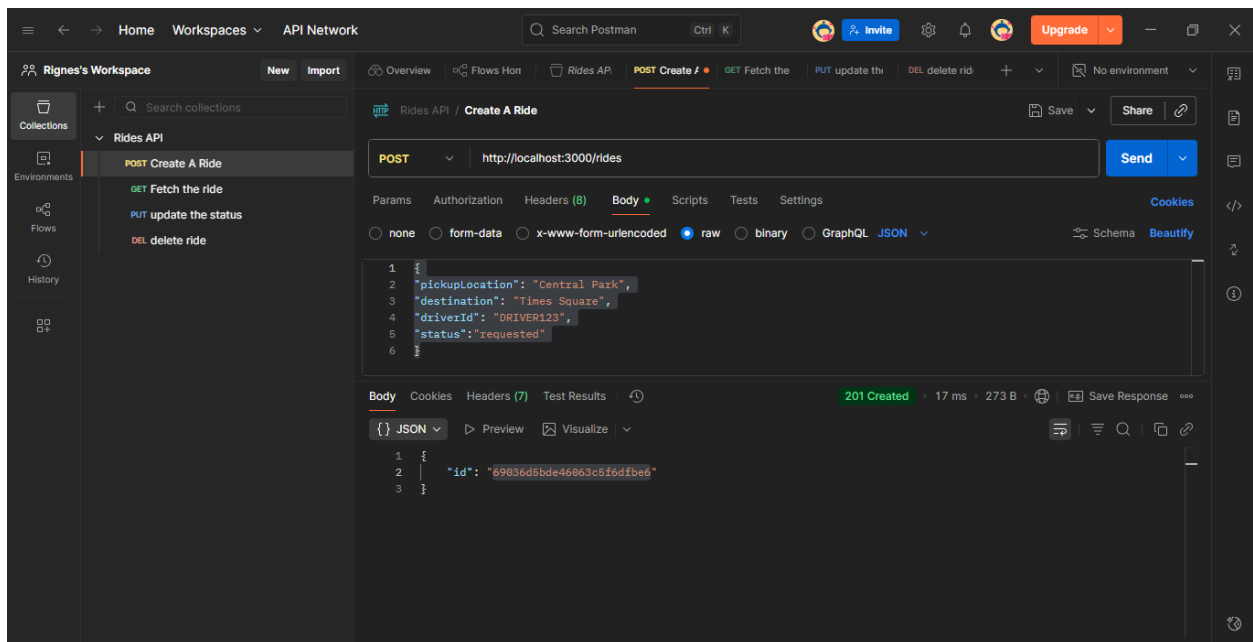


Figure 2: Create a Ride

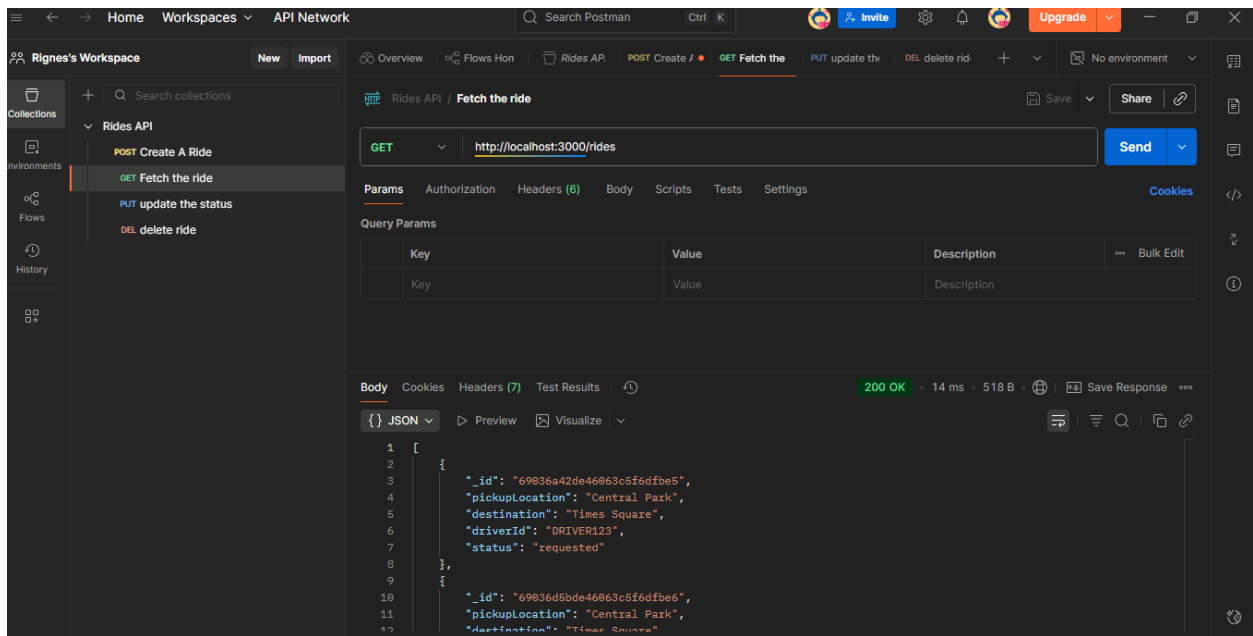


Figure 3: Fetch All Rides

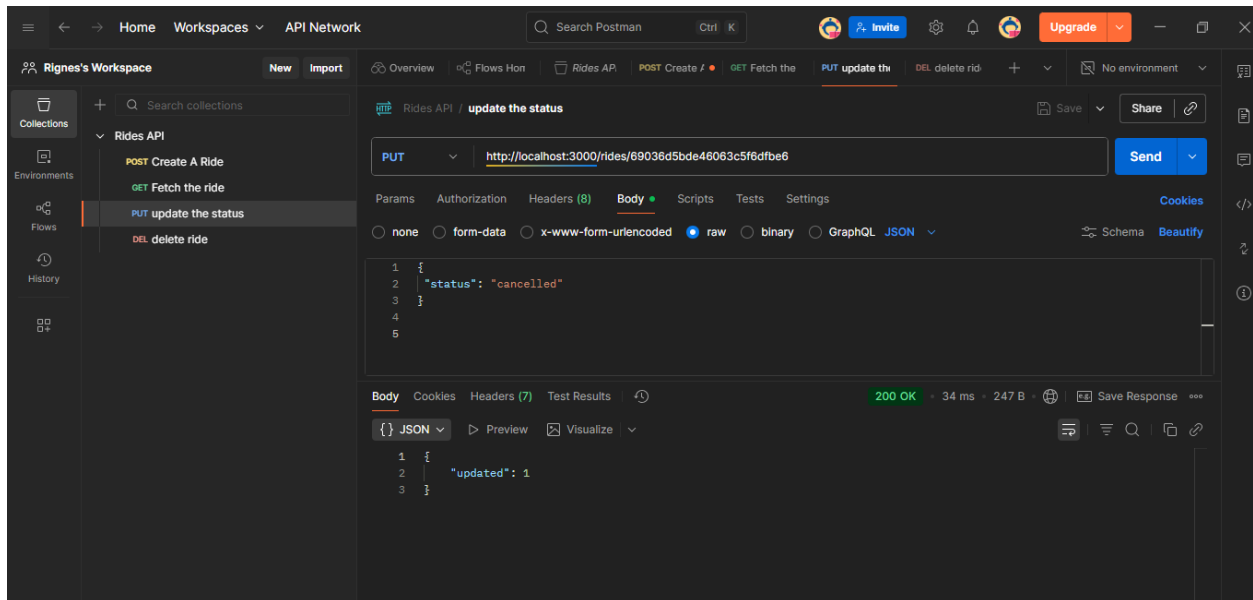


Figure 4: Update Ride Status

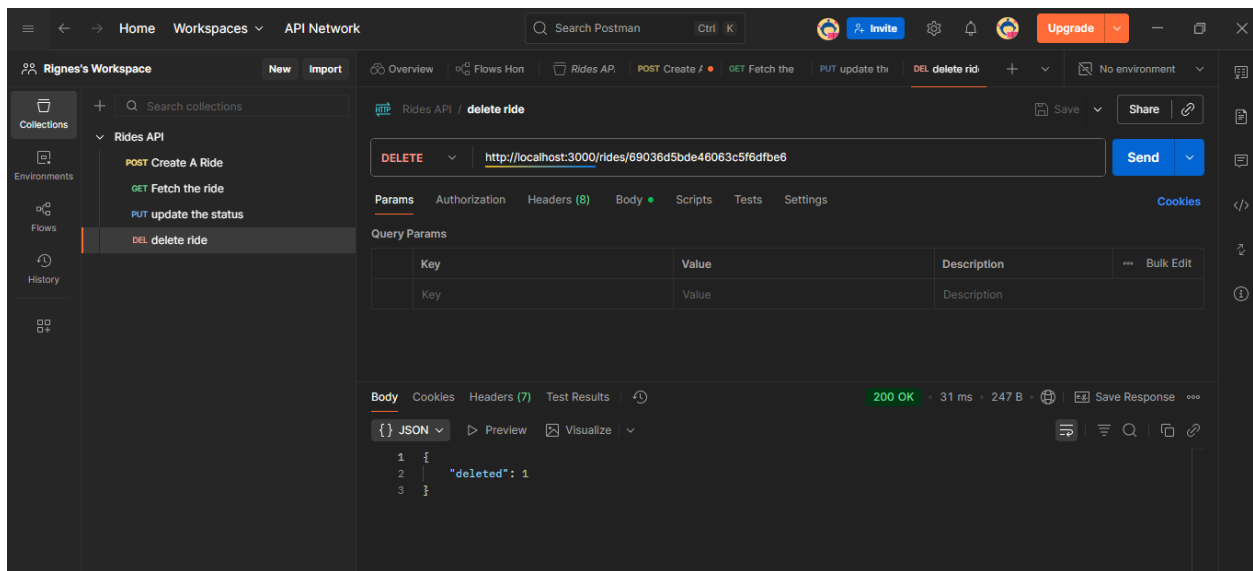


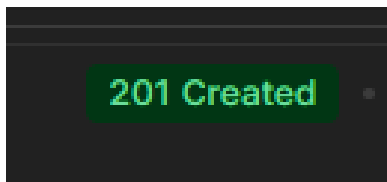
Figure 5: Delete a Ride

LAB EXERCISE

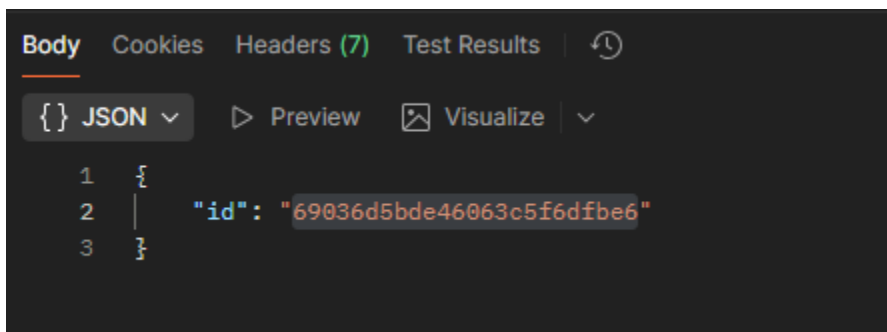
Answer by testing your API in Postman and observing responses.

1. POST Request:

- What HTTP status code is returned when a ride is created successfully?

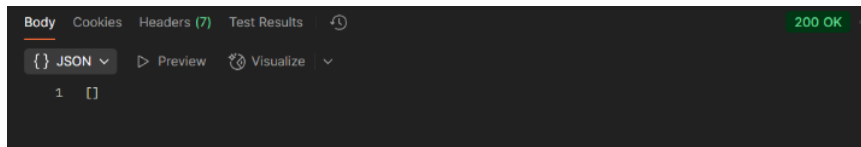


- What is the structure of the response body?

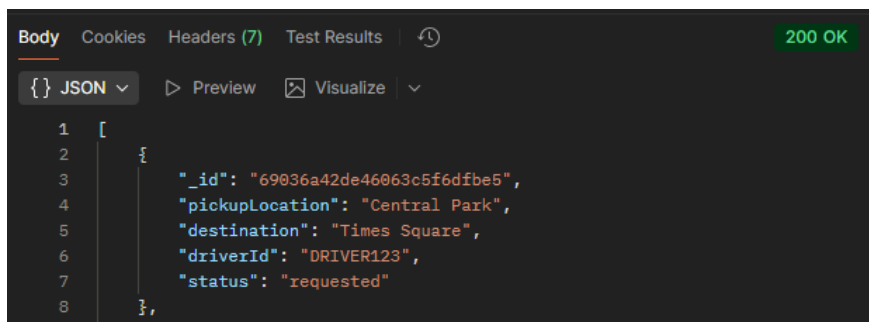


2. GET Request:

- What happens if the rides collection is empty?



- What data type is returned in the response (array/object)?



3. Fix PATCH and DELETE Error:

- Catch the error when requesting PATCH or DELETE API, then try to fix the issue reported.

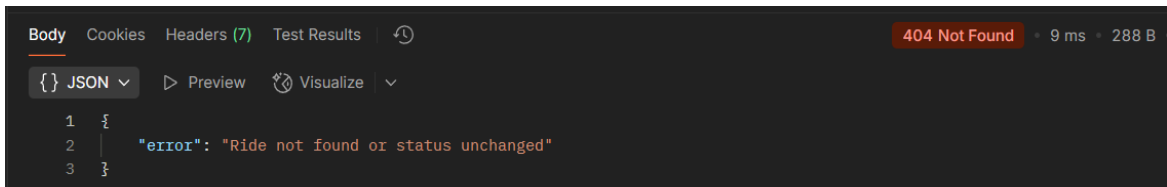
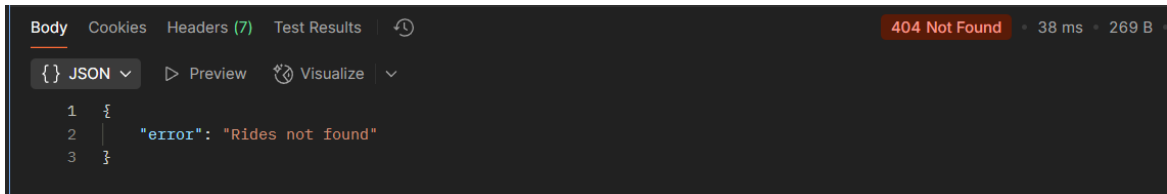
```
// PATCH /rides/:id - UPDATE ride status
app.patch('/rides', async (req, res) => {
  try {
    const result = await db.collection('rides').updateOne(
      { _id: new ObjectId(req.params.id) },
      { $set: { status: req.body.status } }
    );

    if (result.modifiedCount === 0) {
      return res.status(404).json({ error: "Rides not found" });
    }
    res.status(200).json({ updated: result.modifiedCount });
  } catch (err) {
    //HANDLE INVALID ID FORMAT OR DB ERRORS
    res.status(400).json({ error: "Invalid ride ID or data" });
  }
});
```

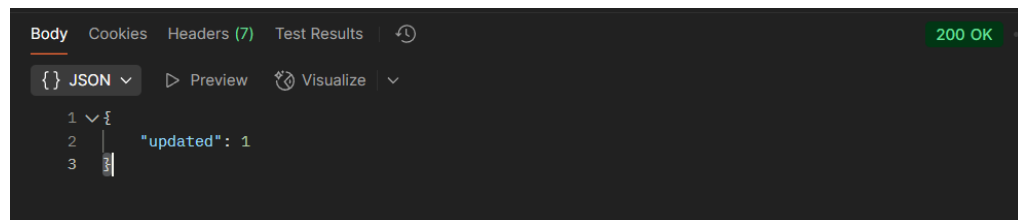
```
//DELETE /rides/:id - Cancel a Ride
app.delete('/rides', async (req, res) => {
  try {
    const result = await db.collection('rides').deleteOne(
      { _id: new ObjectId(req.params.id) },
    );

    if (result.deleteCount === 0) {
      return res.status(404).json({ error: "Rides not found" });
    }
    res.status(200).json({ updated: result.deleteCount });
  } catch (err) {
    res.status(400).json({ error: "Invalid ride ID " });
  }
});
```

- If you try to update a non-existent ride ID, what status code is returned?

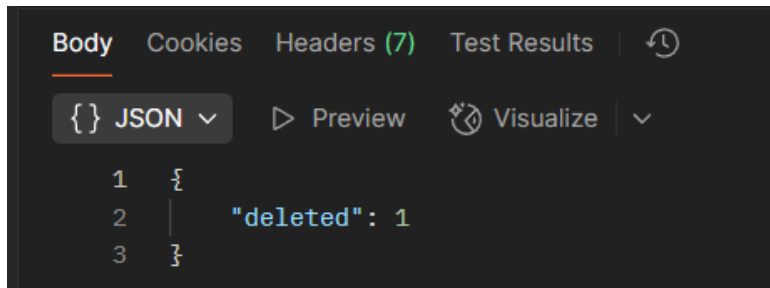


- What is the value of updated in the response if the update succeeds?

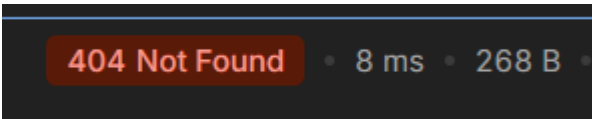
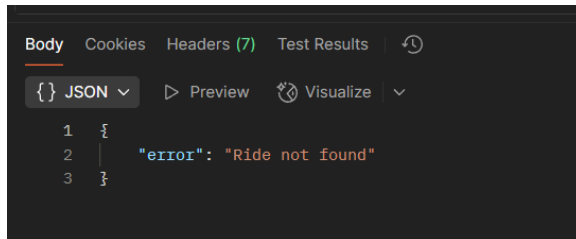


- How does the API differentiate between a successful deletion and a failed one?

If successful:



If the ID didn't match :

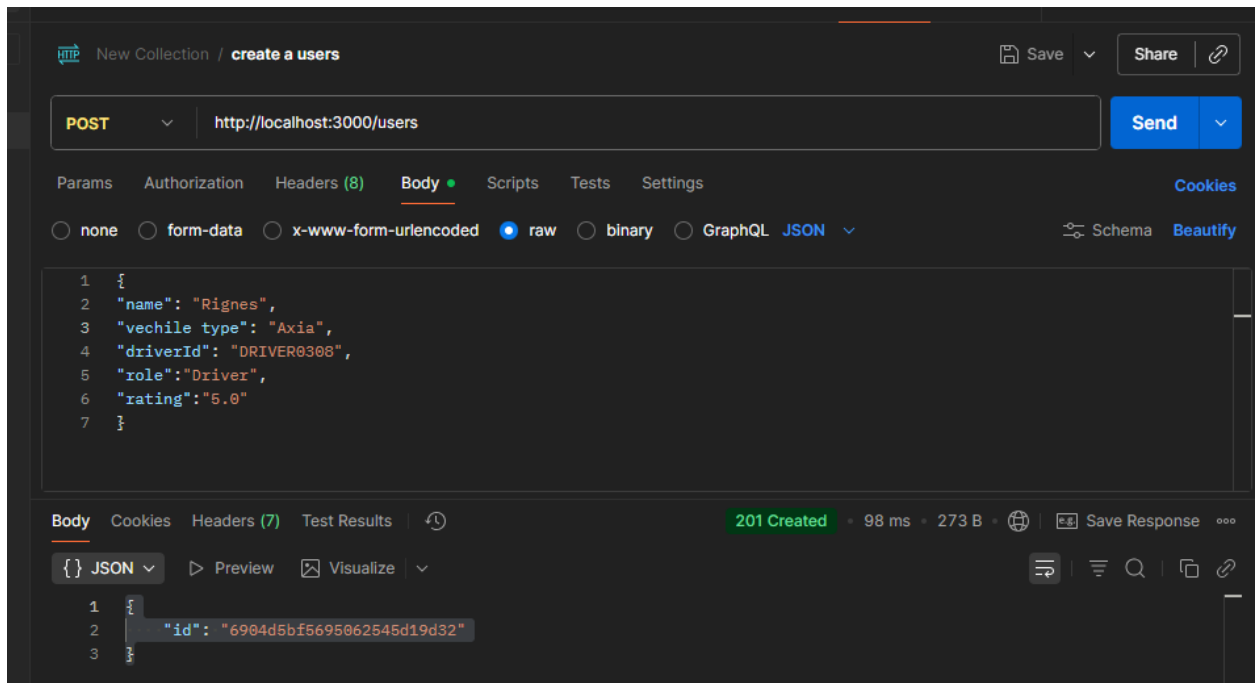


4. Users Endpoints:

- Based on the exercise above, create the endpoints to handle the CRUD operations for users account

I. Create a new users by using POST

```
app.post('/users', async (req, res) => {
  try {
    const result = await db.collection('users').insertOne(req.body);
    res.status(201).json({ id: result.insertedId });
  } catch (err) {
    res.status(400).json({ error: "Invalid user data" });
  }
});
```



II. Read the new users by using GET

```
27 // 1. GET /users - Fetch All users
28 app.get('/users', async (req, res) => {
29   try {
30     const users = await db.collection('users').find().toArray();
31     res.status(200).json(users);
32   } catch (err) {
33     res.status(500).json({ error: "failed to fetch users" });
34   }
35 });
```

GET http://localhost:3000/users Send

Params Authorization Headers (6) Body Scripts Tests Settings Cook

Query Params

	Key	Value	Description	...	Bulk Ed
	Key	Value	Description		

Body Cookies Headers (7) Test Results 200 OK • 19 ms • 365 B • Save Response

{ } JSON Preview Visualize ...

```

1  [
2    {
3      "_id": "6904d5bf5695062545d19d32",
4      "name": "Rignes",
5      "vechile type": "Axia",
6      "driverId": "DRIVER0308",
7      "role": "Driver",
8      "rating": "5.0"
9    }
  ]

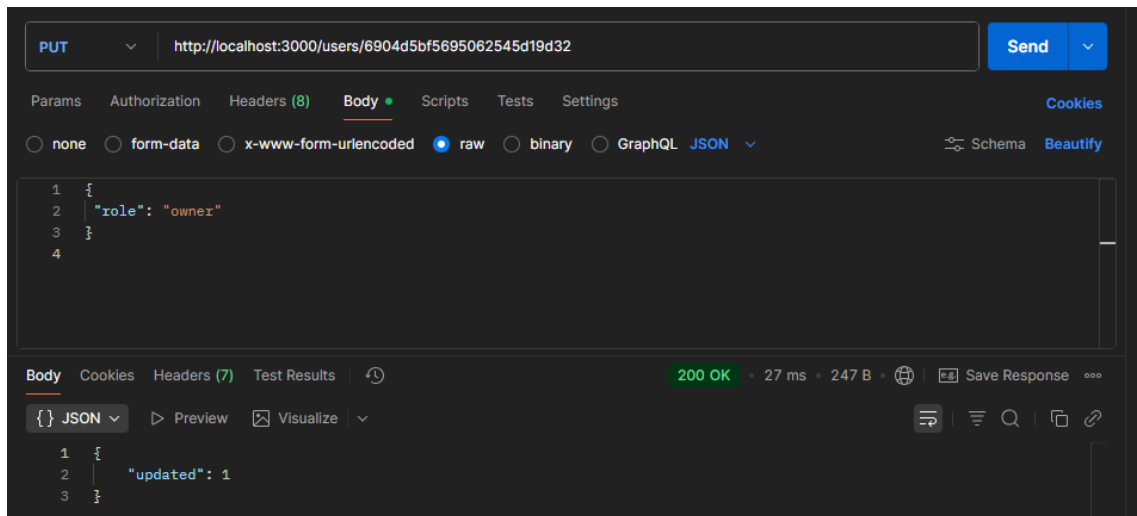
```

III. Update users by ID by using PATCH

```

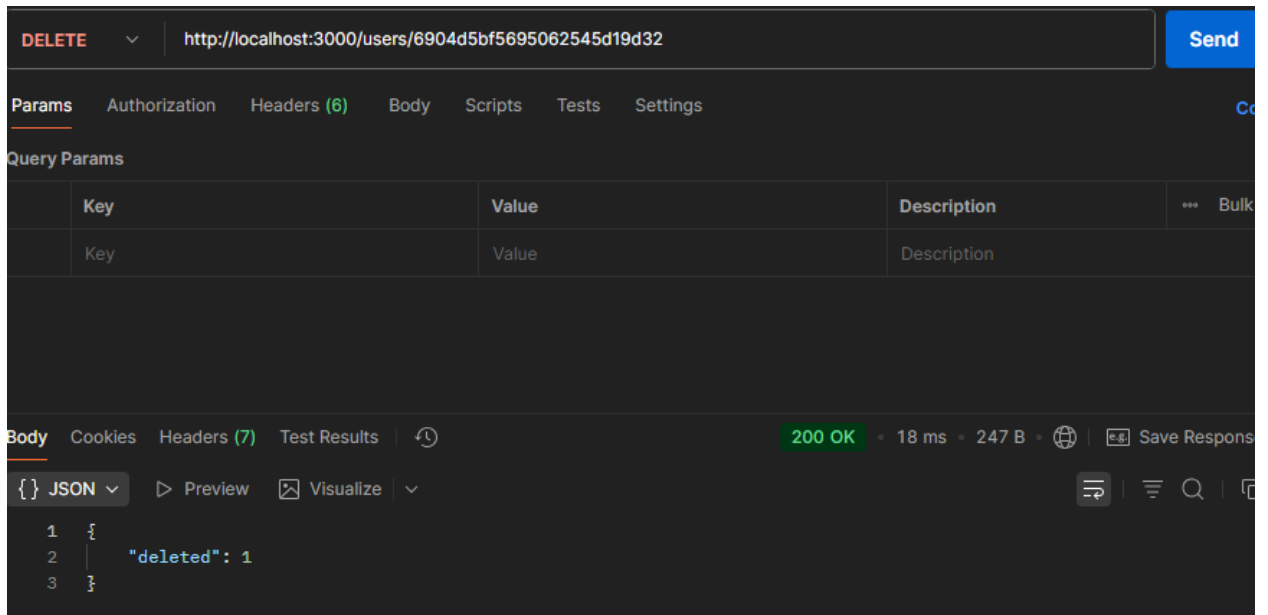
48 app.put('/users/:id', async (req, res) => {
49   try {
50     const result = await db.collection('users').updateOne(
51       { _id: new ObjectId(req.params.id) },
52       { $set: { status: req.body.status } }
53     );
54
55     if (result.modifiedCount === 0) {
56       return res.status(404).json({ error: "user not found" });
57     }
58
59     res.status(200).json({ updated: result.modifiedCount });
60   } catch (err) {
61     res.status(400).json({ error: "Invalid user ID or data" });
62   }
63 });

```



IV. Delete users by ID by using DELETE

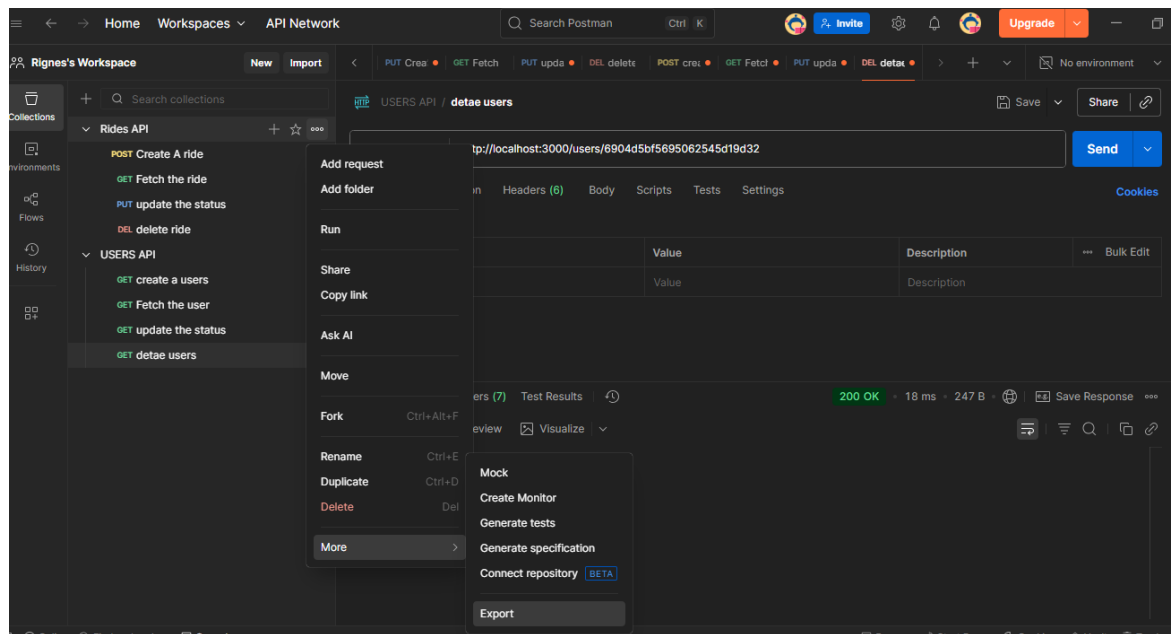
```
66 app.delete('/users/:id', async (req, res) => {
67   try {
68     const result = await db.collection('users').deleteOne(
69       { _id: new ObjectId(req.params.id) }
70     );
71
72     if (result.deletedCount === 0) {
73       return res.status(404).json({ error: "user not found" });
74     }
75
76     res.status(200).json({ deleted: result.deletedCount });
77   } catch (err) {
78     res.status(400).json({ error: "Invalid user ID" });
79   }
80 });
81
```

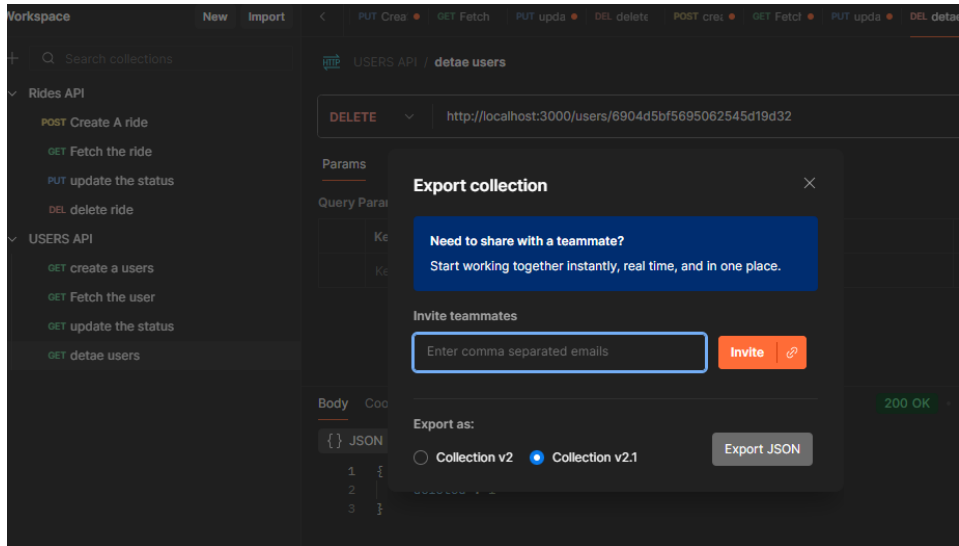


5. FrontEnd:

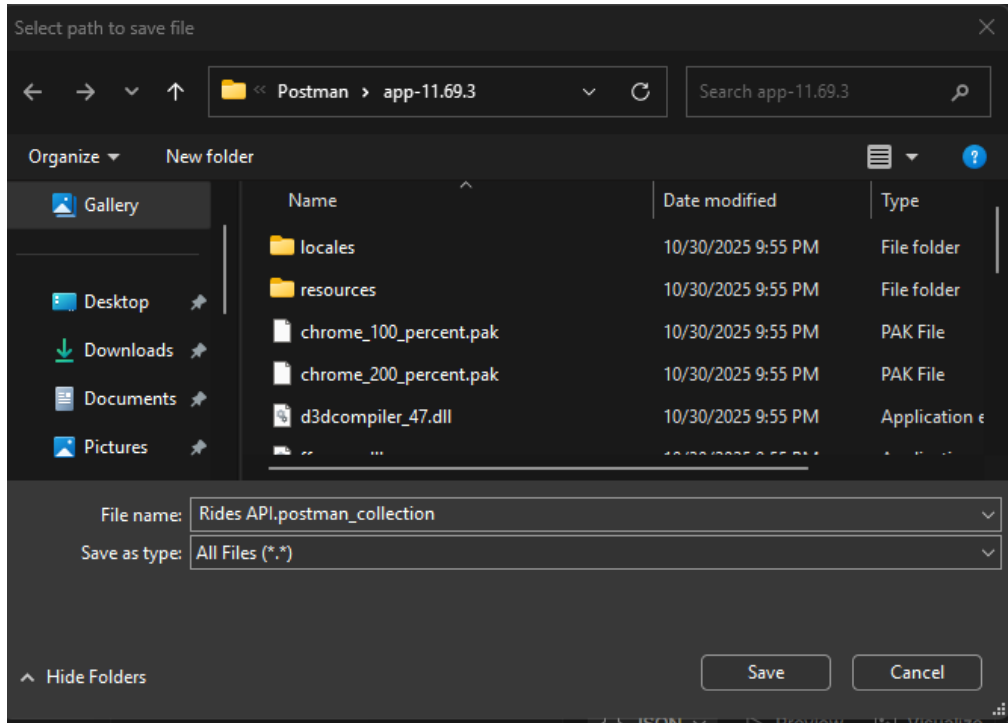
- Upload the Postman JSON to any AI tools, and generate a simple HTML and JS Dashboard for you

i)





ii)



iii)

iv)

> Week2Lab >

Search Week

Sort

View

Name	Date modified	Type	Size
node_modules	11/1/2025 12:50 AM	File folder	
.gitignore	10/23/2025 1:44 PM	Git Ignore Source ...	1 KB
index.js	10/23/2025 3:39 PM	JSFile	4 KB
package.json	11/1/2025 12:50 AM	JSON File	1 KB
package-lock.json	11/1/2025 12:50 AM	JSON File	34 KB
query	10/30/2025 11:36 AM	File	1 KB
README	10/23/2025 3:39 PM	Markdown Source...	0 KB
rides_dashboard (1)	11/1/2025 1:08 AM	Microsoft Edge H...	5 KB
week2	10/23/2025 3:25 PM	Microsoft Edge P...	347 KB
WEEK3.js	11/1/2025 1:00 AM	JSFile	3 KB

v)

Rides Dashboard

This dashboard reads your Postman collection and calls your local API.

API base: http://localhost:3000/rides

ID	Pickup Location	Destination	Driver ID	Status	Actions
69036a42de46063c5f6dfbe5	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete
69037a40de46063c5f6dfbe7	-	-	DRIVER0308	requested	Toggle CancelDelete
69037c7dde46063c5f6dfbe8	-	-	DRIVER0308	requested	Toggle CancelDelete
6904e673d540862d968c27a7	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete
6904e6f0bf1a5e0f0e078aa4	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete
6904e911193d26a9a14ec2bb	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete
6904e9f9193d26a9a14ec2bc	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete
6904ea0e193d26a9a14ec2bd	Central Park	Times Square	DRIVER123	requested	Toggle CancelDelete